Intro2CS 67101-01 Fall Semester Moed Aleph Exam

קראו בזהירות את ההנחיות!

משך הבחינה:
3 שעות, לא כולל תוספת זמן עבור הסטודנטים הזכאים לכך.

מבנה הבחינה:
- **חלק א'** - **10 שאלות** (4 נק' כל אחת). **יש לענות על כל השאלות.**
- **חלק ב'** - **6 שאלות תכנות** (15 נק' כל אחת). **יש לענות רק על 4 מתוך 6 השאלות**. בסוף המבחן תתבקשו לסמן אילו שאלות בחרתם.
- **סימון השאלות אשר בחרתם בבחינה** - **נקודה אחת** בונוס תינתן על סימון השאלות אשר בחרתם בחלק ב', בשאלה נפרדת, בסוף הבחינה. **שימו לב -** לא יהיה ניתן לקבל נקודה זאת בדיעבד, מרגע שהמבחן נסגר, בכל דרך שהיא וגם לא במייל. הקפידו למלא זאת כמה דקות לפני סיום הבחינה.

הנחיות נוספות:
- בשאלות התכנות עליכם לכתוב קוד **יעיל** (מבחינת זמן ריצה וזיכרון) ונקי ככל האפשר. תוכניות לא יעילות או שאינן כתובות היטב לא יקבלו את מלוא הנקודות גם אם הן עובדות נכון.
- בכל שאלות התכנות מותר לכתוב פונקציות עזר נוספות ללא הגבלה. **אך אין לשנות את חתימת הפונקציות (!)**
- אין צורך לתעד את הקוד. אין צורך ב-docstring. הערות לקוד במקומות בהם כתבתם משהו מורכב עשויות לסייע בבדיקת התשובה שלכם אך אינן הכרחיות.
- במידה ותענו על יותר מ-4 שאלות בחלק ב', ייבדקו 4 השאלות הראשונות (במקרה שלא סומנו השאלות לבדיקה).
- **הבחינה היא עם חומר סגור. אין לעשות שימוש בפלאפון מכל סוג שהוא, במחשבון, בדפי נוסחאות, או בכל חומר עזר אחר.**
- מותר לעשות שימוש במחברות טיוטא אשר יוחלקו בכיתות. אין צורך להגיש אותם בסוף הבחינה.
- בשאלות השלמת קוד בחלק א', **אין קשר בין אורך המקטע שמסמן קוד חסר, לבין כמות הקוד שצריך למלא**.
- בשאלות הקוד בחלק ב', היכן שרשום שלא ניתן לערוך את הקוד מעל השורה הנוכחית, **אין לערוך את הקוד מעל השורה הנוכחית.**
- אנא המנעו משאלות לסגל הקורס, אשר אינן קשורות ישירות להבנת השאלה. צוות הקורס לא יוכל לספק רמזים או לענות על שאלות אחרות במהלך המבחן.

בהצלחה!

Q1:

נתונה התוכנית הבאה:

```
def foo(n):
    x=[n]
    def bar():
        return(sum(range(x[0])))
    return bar

boo = foo(n) #line 1
boo()        #line 2
```

מהי סיבוכיות זמן הריצה של השורה המסומנת כ-line 1? תנו חסם הדוק ככל הניתן.

מהי סיבוכיות זמן הריצה של השורה המסומנת כ-line 2? תנו חסם הדוק ככל הניתן.

line1: $O(1)$

line2: $O(n)$

Q2:

נתונה התוכנית הבאה:

```
def frollip(i,j):
    if j==0:
        return sum([frollip(i-1, j+1) for _ in range(i)])
    if i==0:
        return 3
    return frollip(i-1, j+1)
```

מהי סיבוכיות זמן הריצה של הקריאות הבאות כפונקציה של n? תנו חסם הדוק ככל הניתן.

`frollip(n,0)` ⬍

`frollip(n,1)` ⬍

`frollip(1,n)` ⬍

`frollip(0,n)` ⬍

- frollip(n,0): $O(n^2)$
- frollip(n,1): $O(n)$
- frollip(1,n): $O(1)$
- frollip(0,n): $O(1)$

Q3:

נתונים שני משתנים a,b המכילים רשימות של רשימות של מספרים. טיפוסם נתון ע"י:

```
a: List[List[int]]
b: List[List[int]]
```

בכל אחד משני המקרים הבאים, מהו **המספר המינימלי האפשרי** של רשימות **שונות** שמוכלות סה"כ במשתנים a,b יחדיו?
יש לספור רשימות שממש תופסות מקום שונה בזכרון. בספירה יש לכלול כל רשימה אליה ניתן להגיע באופן ישיר או עקיף מהמשתנים a ו- b.
בכל מקרה, כתשובה יש לרשום מספר, או נוסחה שתלוייה בערך n בלבד.

מקרה א': ידוע שמתקיים התנאי `(len(a)==n and len(b)==n and a==b and n>1)`
ישנן לפחות ____ רשימות שונות.

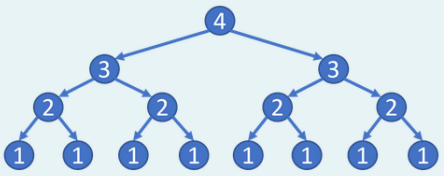
מקרה ב': ידוע שמתקיים התנאי `(len(a)==n and len(b)==n and a!=b and n>1)`
ישנן לפחות ____ רשימות שונות.

- case A: 2
- case B: 4

Q4:

נגדיר "עץ מספרים בינארי מסדר n", עבור מספר שלם אי-שלילי n, להיות עץ בגובה n-1, שבו לכל צומת שאינה עלה יש שני בנים.
הערך בשורש הוא n, הערך בכל אחד משני הבנים הוא n-1, הערך בכל אחד מהבנים שלהם הוא n-2 וכך הלאה.

לדוגמא, באיור מתואר "עץ מספרים בינארי מסדר 4":



בקטע הקוד המצורף מוגדרת מחלקה TreeNode עבור צומת בעץ. מוגדרת גם הפונקציה f, שמקבלת מספר n.
השלימו את הפונקציה f (מקטע הקוד החסר נמצא מתחת למקרה ה else) כך שתיצור עץ מספרים בינארי מסדר n ותחזיר את השורש שלו.
שימו לב, כל צומת בעץ צריך להיות אובייקט שונה מצמתים אחרים.

```
class TreeNode:
    def __init__(self, num: int,
                 left: Optional["TreeNode"], right: Optional["TreeNode"]):
        self._num = num
        self._left = left
        self._right = right


def f(n) -> Optional[TreeNode]:
    if n == 0:
        return None
    else:
        # your code goes here
```

הקוד החסר הוא:

```
return TreeNode(n, f(n-1), f(n-1))
```

Q5:

נתונה רשימה (list) המכילה מספרים, `lst`. ברצוננו להפוך אותה ל**רשימה מקושרת** בסדר הפוך. כלומר, עבור הקלט

`[1, 2, 3, 4, 5]`

נקבל רשימה מקושרת:

`5 -> 4 -> 3 -> 2 -> 1 -> None`

נתון הקוד הבא לביצוע הפעולה. הקוד מגדיר את המחלקה Node המתארת איבר ברשימה. הגדירו את הפונקציה `f1` ש `reduce` מקבלת כקלט, כדי להפוך את הרשימה `lst` לרשימה מקושרת כמו שמתואר מעלה.

בתשובתכם כיתבו אך ורק את הקוד שמחליף את סימני השאלה.

```
from functools import reduce

class Node:
    def __init__(self, num, next) -> None:
        self.num = num
        self.next = next

lst = [1, 2, 3, 4, 5]
f1 = ???
head = reduce(f1, lst, None)
```

הקוד החסר הוא:

```
f1 = lambda a,x: Node(x,a)
```

Q6:

אלגוריתם מיון נקרא **"יציב" (stable)** אם הוא שומר על הסדר של איברים שווים ברשימה.

כלומר, בהינתן שהאלגוריתם ממיין לפי יחס סדר מסויים, אם יש שני איברים ברשימה שנחשבים לשווים זה לזה ע"י יחס הסדר, אזי אם איבר a הופיע משמאל לאיבר b ברשימה לפני הפעלת המיון, אז הוא **תמיד** יופיע משמאל לאיבר b גם לאחר המיון.

<u>לדוגמא</u>, אם נתונה רשימה של אובייקטים המייצגים סטודנטים וממיינים אותם לפי ציון המבחן שלהם, הסדר בין סטודנטים בעלי אותו ציון, ישמר.

כעת נתונים שני אלגוריתמי מיון (שניהם ממומשים בצורה נכונה). **לגבי כל אלגוריתם, סמנו אם הוא יציב או לא.**

```
def sort1(lst):
    for ind in range(len(lst)):
        min_index = ind
        for j in range(ind + 1, len(lst)):
            if lst[j] < lst[min_index]:
                min_index = j
        lst[ind], lst[min_index] = lst[min_index], lst[ind]
```

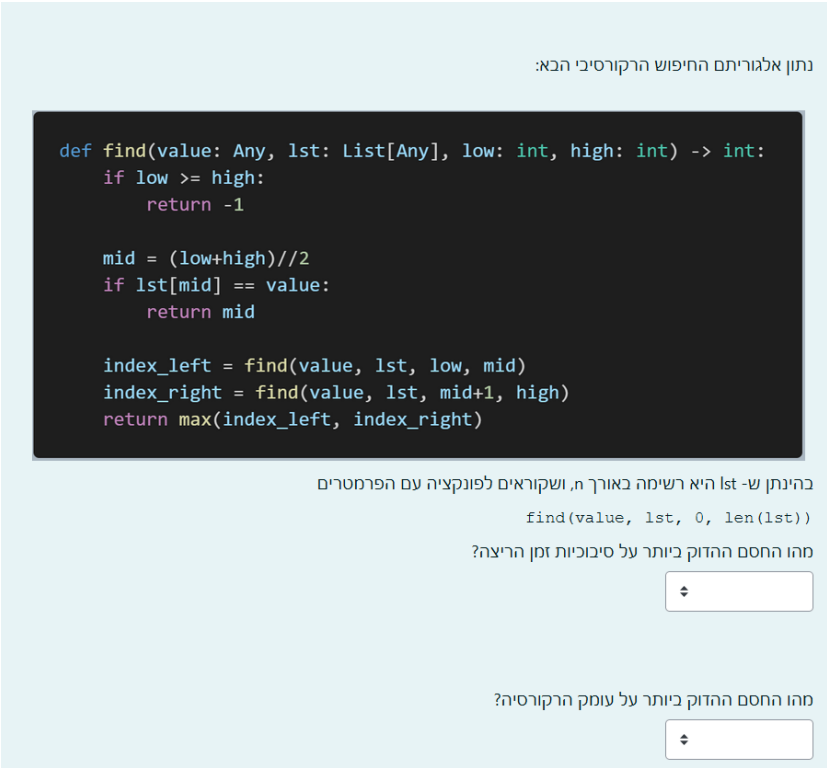
```
def sort2(lst):
    n = len(lst)
    for i in range(n):
        for j in range(0, n-i-1):
            if lst[j] > lst[j+1]:
                lst[j], lst[j+1] = lst[j+1], lst[j]
```

sort1 is

sort2 is

- sort1 is not stable
- sort2 is stable

Q7:



נתון אלגוריתם החיפוש הרקורסיבי הבא:

```python
def find(value: Any, lst: List[Any], low: int, high: int) -> int:
    if low >= high:
        return -1

    mid = (low+high)//2
    if lst[mid] == value:
        return mid

    index_left = find(value, lst, low, mid)
    index_right = find(value, lst, mid+1, high)
    return max(index_left, index_right)
```

בהינתן ש- lst היא רשימה באורך n, ושקוראים לפונקציה עם הפרמטרים

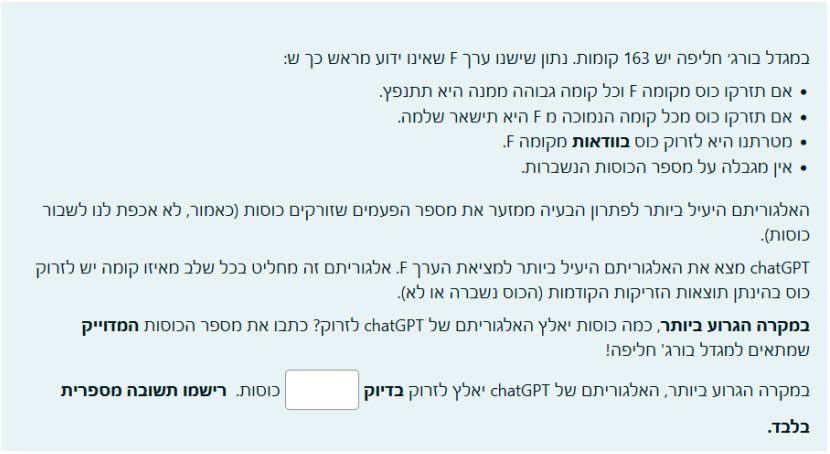
```
find(value, lst, 0, len(lst))
```

מהו החסם ההדוק ביותר על סיבוכיות זמן הריצה?

מהו החסם ההדוק ביותר על עומק הרקורסיה?

- Runtime complexity: $O(n)$
- Recursion depth: $O(log(n))$

Q8:



במגדל בורג' חליפה יש 163 קומות. נתון שישנו ערך F שאינו ידוע מראש כך ש:

- אם תזרקו כוס מקומה F וכל קומה גבוהה ממנה היא תתנפץ.
- אם תזרקו כוס מכל קומה הנמוכה מ F היא תישאר שלמה.
- מטרתנו היא לזרוק כוס **בוודאות** מקומה F.
- אין מגבלה על מספר הכוסות הנשברות.

האלגוריתם היעיל ביותר לפתרון הבעיה ממזער את מספר הפעמים שזורקים כוסות (כאמור, לא אכפת לנו לשבור כוסות).

chatGPT מצא את האלגוריתם היעיל ביותר למציאת הערך F. אלגוריתם זה מחליט בכל שלב מאיזו קומה יש לזרוק כוס בהינתן תוצאות הזריקות הקודמות (הכוס נשברה או לא).

במקרה הגרוע ביותר, כמה כוסות יאלץ האלגוריתם של chatGPT לזרוק? כתבו את מספר הכוסות **המדוייק** שמתאים למגדל בורג' חליפה!

במקרה הגרוע ביותר, האלגוריתם של chatGPT יאלץ לזרוק **בדיוק** כוסות. **רשמו תשובה מספרית בלבד.**

An optimal algorithm would need exactly 8 throws in the worst case.

Q9:



מימשתם את אלגוריתם quicksort בצורה הרגילה (כפי שמוצג פה) חוץ משינוי אחד:

```
def quick_sort(data):
    _quick_sort_helper(data, 0, len(data))

def _quick_sort_helper(data, low, high):
    if low < high-1:
        pivot_index = partition(data, low, high)
        _quick_sort_helper(data, low, pivot_index)
        _quick_sort_helper(data, pivot_index+1, high)
```

בפונקציה `partition` ערך `pivot` נבחר **ממרכז הרשימה ולא באקראי**:

```
pivot =  (low + high)//2
```

הניחו שהפונקציה partition מפרידה בין ערכים גדולים מה-pivot וערכים קטנים ממנו (בדיוק כפי שלמדנו בשיעור) ושמלבד זאת היא לא משנה את הסדר בין ערכים ברשימה. כלומר, ערכים קטנים מה-pivot שומרים על אותו סדר יחסי וערכים גדולים מה-pivot שומרים על אותו סדר יחסי שהיה להם ברשימה המקורית.

מצאו פרמוטציה של הרשימה [0,1,2,3,4,5,6,7,8,9] שתדרוש את זמן הריצה הגבוה ביותר לאלגוריתם המיון שהגדרנו.

כתבו את הפרמוטציה בפורמט של רשימה בפייתון. לדוגמא: [0,1,2,3,4,5,6,7,8,9]

One such permutation is: [9,7,5,3,1,0,2,4,6,8]

Q10



נתונה הפונקציה הבאה:

```
def f(n: int) -> int:
    if n <= 4:
        return n
    else:
        return f(n-1) + f(n-2) + f(n-3) + f(n-4)
```

מהו עומק הרקורסיה כפונקציה של n? (הניחו כי n חיובי גדול מ-4). רשמו נוסחה מדוייקת התלויה ב-n. יש לספור גם את הקריאה ההתחלתית ל-f.

מהי סיבוכיות זמן הריצה כפונקציה של n?

בחרו בחסם ההדוק ביותר.

- Recursion depth for n>4: $n-3$
- We decided to accept: $O(4^n)$, $O(3^n)$, $O(2^n)$, $O(e^n)$

Seeing which bound is tightest is hard. It is realatively easy to prove that $O(4^n)$ is a bound, other bounds are also correct. $O(2^n)$ is the tightest one.

Q11:

ממש את הדקורטור cache שיכול לקשט פונקציה כללית (בלי להניח את מספר הארגומנטים שלה או את שמם) כך ש:

1. הפונקציה המקושטת מחזירה את הערך של הפונקציה המקורית כרגיל כל עוד לא ניתן ה keyword argument: use_cache = True אבל היא שומרת את תוצאות החישוב של קריאות שבוצעו עבור שימוש עתידי (ראו מטה).
2. לאחר שניתן ה keyword argument כפי שנכתב מעלה (כלומר, הפונקציה נקראה עם use_cache = True) ועד שהפונקציה נקראת עם הארגומנט use_cache=False, מופעל מנגנון ה-cache. כל עוד המנגנון פועל, הפונקציה תחזיר תשובות ששמרה מהרצות בעבר אם כבר נקראה עם ארגומנטים זהים (מלבד הארגומנט use_cache, שאין להתחשב בו למטרות הבדיקה).

<u>הבהרות:</u>

- מנגנון ה-cache שומר פנימית תוצאות של חישוב ללא קשר להפעלת use_cache=True. למעשה ארגומנט זה רק מאפשר לפונקציה לעשות שימוש בערכים השמורים ולהחזירם ללא הרצת הפונקציה המקורית.
- בכל מקרה, אם הארגומנטים חדשים ולא נראו בעבר, יש להריץ את הפונקציה המקורית.
- כאשר נקראת הפונקציה עם use_cache=False היא חוזרת להתנהג כמו בסעיף 1.
- אסור לעשות שימוש במודול functools בשאלה זו ובפרט בדקורטור cache המובנה של פייתון ממודול זה.
- ניתן להניח שלפונקציה המקושטת יקראו ללא כל keywords מלבד use_cache ושכל הפרמטרים של הפונקציה הם immutable, hashable.

לדוגמא:

```
@cache
def f(n):
    sum = 0
    for i in range(n):
        sum += i**2
    return sum

f(1000) # runs 1000 loop iterations
f(100) # runs 100 loop iterations
f(1000, use_cache = True) # returns without running any loop iterations
f(100) # returns without running any loop iterations
f(1000, use_cache = False) # runs 1000 loop iterations
f(100) # runs 100 loop iterations
```

In [ ]:
```
def cache(func):
    returned = {}
    use_cache = [False]
    def wrapper(*args, **kwargs):
        if "use_cache" in kwargs:
            use_cache[0] = kwargs["use_cache"]

        if use_cache[0] and args in returned:
            return returned[args]

        result = func(*args)
        returned[args] = result
        return result
    return wrapper
```

Q12:

אסור לייבא בשאלה זו מודולים חיצוניים (מלבד אולי typing). יש לעשות שימוש בפעולות חשבון פשוטות (כפל, חיבור, חיסור, חילוק, חילוק בשלמים , ומודולו)

כתבו פונקציה בשם `exponent(x,n)` המחזירה את תוצאת העלאת המספר `x` בחזקת `n`.

לדוגמא, הקריאה `exponent(2,3)` תחזיר `8` (כי 2 בחזקת 3 שווה ל-8).

<u>שימו לב:</u>

- יש לפתור את השאלה **ביעילות המיטבית**. פתרונות ביעילות נמוכה יאבדו את מרבית הנקודות.
- **אסור לעשות שימוש ברקורסיה** בפתרון (עשו שימוש בלולאות!).
- מותר לעשות שימוש בפעולות חיבור וכפל עבור מספרים, אבל **אסור** כמובן לעשות שימוש בפעולות חזקה שבנויות בפייתון.
- המספר `x` הוא מספר כלשהוא מטיפוס `int` או `float`.
- המספר `n` הוא מטיפוס `int` והוא אי-שלילי. אין צורך לטפל במקרה של `n` שלילי.

In [ ]:
```python
from typing import Union

def exponent(x: Union[int,float], n:int) -> Union[int,float] :
    res = 1
    while n>0:
        if n % 2:
            res *= x
        n = n//2
        x *= x
    return res
```

Q13:

נגדיר הרחבה של סדרת פיבונאצ'י שאנו קוראים mb_fib.

הסדרה הזו מוגדרת על ידי הנוסחה המתמטית: $f_n = f_{n-1} \cdot f_{n-2} \cdot f_{n-3}$

האיברים הראשונים בסדרה הם: 1,2,3,6,36

כתבו פונקציה המחשבת את האיבר מספר n בסדרה: mb_fib(n)

- האיבר הראשון בסדרה הוא עבור n=1. כלומר, mb_fib(1)==1
- השתמשו ברקורסיה.
- **אין להשתמש בלולאות בשום מקום בקוד.**
- **אסור לעשות שימוש במודול functools.**
- **אסור לעשות שימוש במבני נתונים שגודלם יותר מ-O(1).**
- **יש לרוץ ביעילות המיטבית.** פתרונות ביעילות פחותה יאבדו את מירב הנקודות.
- כמובן מותר להשתמש בפונקציות עזר (כמו בכל שאלה אחרת בבחינה).

In [ ]:
```python
def mb_fib(m: int) -> int:
    return m if m<4 else fib_helper(m)[2]

def fib_helper(m):
    if m==3:
        return 1,2,3
    xm3, xm2, xm1 = fib_helper(m-1)
    return  xm2, xm1, xm1 *  xm2 * xm3
```

Q14:

ניתן לייצג מחרוזת על ידי רשימה מקושרת באופן הבא:

כל איבר ברשימה המקושרת מכיל תו בודד, ושרשור התווים של אברי הרשימה המקושרת לפי הסדר שלהם (מראש הרשימה המקושרת, עד לסופה) הוא המחרוזת.

נרחיב את הייצוג הנ"ל כך שכל איבר ברשימה גם מכיל מספר שלם וחיובי (בנוסף לתו) המייצג את מספר הפעמים שהתו חוזר במחרוזת ברצף, ראו את מימוש Node למטה. לדוגמה, הרשימה המקושרת:

[data="c", rep=2] -> [data="a", rep=1] -> None

מייצגת את המחרוזת "cca"

כתבו את הפונקציה `compress(head: Node)` בקוד למטה המקבלת מחרוזת בייצוג הנ"ל. על הפונקציה שתכתבו לדחוס את הרשימה שניתנה לה, כך שרצף של איברים המכילים את אותו תו יוחלף באיבר אחד המייצג את המחרוזת עם מספר חזרות מתאים.

<u>לדוגמה</u>, הפונקציה תהפוך את הרשימה המקושרת הבאה:

[data="c", rep=2] -> [data="a", rep=3] -> [data="a", rep=1] -> [data="a", rep=3] -> None

לרשימה המקושרת הזו:

[data="c", rep=2] -> [data="a", rep=7] -> None

שימו לב:

- **אסור** לפונקציה החדשה ליצור רשימה מקושרת חדשה או איברי רשימה חדשים, עליה לשנות את הרשימה שניתנת לה כקלט.
- ניתן להניח שהרשימה לא ריקה, כלומר head איננו None
- אסור לעשות שימוש ביותר מ-O(1) זכרון נוסף (מלבד איברי הרשימה עצמה).
- הפונקציה מחזירה None. ראש הרשימה המקושרת הדחוסה הוא אותו Node שהועבר כפרמטר head.
- מותר לגשת לשדות הרשימה ישירות. אין צורך בפונקציות getter או setter

In [ ]:
```python
from typing import Optional

class Node: # DO NOT CHANGE THIS CLASS!
    def __init__(self, data: str, rep: int, next_node: Optional["Node"]) -> None:
        self.data = data
        self.rep = rep
        self.next = next_node

# DO NOT EDIT CODE ABOVE THIS LINE! (CODE ABOVE THIS LINE WAS PROVIDED)

def compress(head: Node) -> None:
    while head is not None:
        while head.next is not None and head.data == head.next.data:
            head.rep += head.next.rep
            head.next = head.next.next
        head = head.next
```

Q15:

מטרתכם להושיב n אנשים ב k שולחנות. כאשר n ו-k הם מספרים חיוביים מסוג `int` (גדולים מ-0). כל אדם מזוהה על ידי מספר שלם מטיפוס `int` בין 1 ל n (כולל).

- קיים מבנה נתונים hate שמתאר עם אילו אנשים כל אדם אינו מעוניין לשבת בשולחן. `hate` הוא משתנה מטיפוס `set` המכיל טאפלים (tuples) שמתארים את יחסי הדחיה בין אנשים:
  אדם מספר i לא מעוניין לשבת בשולחן עם אדם מספר j אם ורק אם הטאפל `(i,j)` נמצא בקבוצה `hate`.

כתבו פונקציה `assign` המקבלת את `n, k, hate` ומחפשת דרך להושיב את כל האנשים, כך שבכל שולחן אין שני אנשים שלא מעוניינים לשבת יחד.

- אם לא קיים סידור חוקי, הפונקציה תחזיר `None`.
- אחרת, הפונקציה מחזירה רשימה המכילה k רשימות. כל אחת מ-k הרשימות תכיל את מספריהם של האנשים היושבים בשולחן מסויים.

חתימת הפונקציה היא:

```
def assign(n: int, k: int, hate: Set[Tuple[int, int]]) ->
Optional[List[List[int]]]:
```

דגשים:

- בערך ההחזרה של הפונקציה, אין חשיבות לסדר האנשים בכל שולחן ואין חשיבות לסדר השולחנות.
- אין מגבלה על מספר האנשים שמותר להושיב בכל שולחן.
- אין להניח שיחסי האיבה הדדיים, כלומר יכול להיות שמתקיים:

```
(i,j) in hate and (j,i) not in hate
```

- שימו לב: יש להשתמש בbacktracking כדי למצוא סידור חוקי של האנשים אם קיים אחד כזה.

דוגמא:

אם קובעים ש- `hate1 = {(1,2), (1,3)}`
אסור להושיב יחד את אנשים 1,2 ואסור להושיב יחד את אנשים 1,3. לכן הקריאה: `assign(4, 2, hate1)` יכולה להחזיר למשל `[[3 ,2] ,[4 ,1]]` כלומר להושיב יחד את אנשים 1,4 בשולחן אחד ולהושיב יחד את 2,3 בשולחן אחר.

לעומת זאת, אם קובעים ש- hate1 = {(1,2), (1,3), (2,3)} אזי לא ניתן להושיב את 4 האנשים ב-2 שולחנות (כי 1,2,3 חייבים להיות כל אחד בשולחן אחר וצריך לפחות 3 שולחנות).

In [ ]:
```python
from typing import Optional, List, Set, Tuple

def assign(n: int, k: int, hate: Set[Tuple[int, int]]) -> Optional[List[List[int]]]:
    sol: List[List[int]] = [[] for _ in range(k)]
    return sol if helper(n, k, hate, sol, 1) else None


def helper(n: int, k: int, hate: Set[Tuple[int, int]],
           sol: List[List[int]], cur_person_ind: int) -> bool:
    if cur_person_ind == n+1:
        return True
    for table in sol:
        if can_sit(cur_person_ind, table, hate):
            table.append(cur_person_ind)
            if helper(n, k, hate, sol, cur_person_ind+1):
                return True
            table.pop()
    return False


def can_sit(person_ind: int, table: List[int], hate: Set[Tuple[int, int]]) -> bool:
    for person in table:
        if (person, person_ind) in hate or (person_ind, person) in hate:
            return False
    return True
```

Q16:

כתבו מחלקה `TextDoc` ממנה ניתן ליצור אובייקטים המייצגים מסמך טקסט הכולל סמן (באנגלית, "סמן" == "caret"). כשמקלידים תווים חדשים למסמך, הם מוקלדים במיקום הסמן, והסמן מתקדם קדימה עם ההקלדה.

למחלקה צריכות להיות המתודות הבאות:

- `doc = TextDoc() # מייצר מסמך ריק`
- `doc.move_caret(row, col) #מזיז את הסמן במסמך לשורה ולעמודה הנתונות`
- `doc.type_in(char)  # מקליד תו בודד למסמך`
- `doc.get_current_line()  # מחזיר כמחרוזת את תוכן השורה בה נמצא הסמן`

<u>פרטים נוספים לגבי המתודות:</u>

- המתודות `type_in` ו- `move_caret` מחזירות `None`.
- בעת יצירת מסמך חדש, הסמן מתחיל בשורה 0 בעמודה 0, והשורה הנוכחית היא בעצם מחרוזת ריקה.
- הפרמטרים `row` ו- `col` במתודה `move_caret` נתונים כשני מספרים מטיפוס `int`. אם מבקשים מעבר לעמודה ושורה שלא קיימות במסמך, יש לזרוק שגיאה מסוג `ValueError` (שגיאה כזו כבר מוגדרת בפייתון).
- שימו לב שבשורה שבה יש k תווים רגילים (שאינם תו ירידת השורה) אפשר לשים את הסמן בעמודות 0 עד k (כולל עמודה k שהיא אחרי התו האחרון).
- ניתן להניח שהתווים שמקלידים במתודה `type_in` יהיו או תווי אותיות ומספרים רגילים (באנגלית) או תו ירידת שורה "`\n`". אין צורך לבדוק את נכונות התווים. ניתן להניח שבכל פעם יוקלד תו יחיד.
  - במקרה של הקלדת תו ירידת שורה, תפוצל השורה המקורית לשתי שורות בנקודה בה היה הסמן. הסמן ירד שורה ויהיה בעמודה 0 של השורה החדשה.
  - במקרה שהוקלד תו רגיל, הוא יוכנס למסמך במיקום הסמן, לפני תווים הממוקמים אחרי הסמן.
- המחרוזות שחוזרות מ-`get_current_line` אינן כוללות תווי ירידת שורה.
- במימוש זה, אין אפשרות למחוק תווים או שורות במסמך.
- בעת כתיבת המחלקה, יש לתת עדיפות ליעילות המיטבית של המתודות `move_caret` ו- `get_current_line` על פני מתודות אחרות.

In [ ]:

```
from typing import Union

class TextDoc:
    def __init__(self):
        self._lines = [""]
        self._line = 0
        self._col = 0

    def move_caret(self, row: int, col: int) -> None:
        if row >= len(self._lines) or  row < 0 or col < 0 or col > len(self._lines[row]):
            raise ValueError(f"Attempted to move caret to bad location: {row}, {col}")
        self._line = row
        self._col = col


    def type_in(self, character:str) -> None:
        line1 = self._lines[self._line][:self._col]
        line2 = self._lines[self._line][self._col:]
        if character == "\n":
            self._lines[self._line] = line1
            self._lines.insert(self._line+1, line2)
            self._line += 1
            self._col = 0
        else:
            self._lines[self._line] = line1 + character + line2
            self._col+=1


    def get_current_line(self) -> str:
        return self._lines[self._line]
```